**PROJECT REPORT**

# AI + GAMING – INTELLIGENT GAMEPLAY AND PLAYER EXPERIENCE SYSTEM

Submitted by

MUTHU KUMARAN M
REG NO :- 711525BCS099

**BACHELOR OF ENGINEERING**
in Computer Science and Engineering

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
**(KIT-CBE)**

## ABSTRACT

AI + Gaming is a proposed intelligent gaming system that uses artificial intelligence to improve gameplay, adapt difficulty, and provide useful player insights. Traditional games often use fixed difficulty and predictable non-player character (NPC) behaviour. The proposed system analyses game events and player actions to support adaptive challenges, smarter NPC decisions, and engaging gameplay. Player profiles and session statistics help track progress.

The project demonstrates Object-Oriented Programming concepts using Java. Classes such as Player, GameSession, GameAI, NPC, and GameAIService represent game entities and responsibilities. Encapsulation protects player and session data, inheritance models related game characters,

polymorphism supports different AI behaviours, and abstraction separates game logic from implementation. A relational database can store player profiles, scores, and session results. Fairness and privacy are central to the design. Player data should be collected only when needed, and game actions should be validated by the application. Authentication, role-based access, secure database operations, anti-cheat checks, and activity logs help protect accounts and maintain fair gameplay. This report presents the system design, OOPS mapping, database structure, core Java code, testing plan, limitations, and future improvements.

## REPORT CONTENTS


| Section | Topics |
|---|---|
| 1 | Introduction, problem statement, objectives |
| 2 | Existing and proposed systems; requirements |
| 3 | Architecture and workflow |
| 4 | OOPS concepts and class design |
| 5 | Database design |
| 6 | Java implementation |
| 7 | AI code, security, and testing |
| 8 | Advantages, limitations, future scope, conclusion |
| 9 | References and execution notes |


## 1. INTRODUCTION

## 1.1 Background

Many games use fixed difficulty levels and scripted non-player character behaviour. This can make gameplay repetitive and may not suit players with different skill levels. An AI-supported game can analyse gameplay events and adapt challenges, enemy strategies, and in-game guidance.

## 1.2 Problem Statement

Fixed game rules, predictable NPCs, and limited feedback can reduce replay value and player engagement. AI + Gaming proposes a Java-based prototype that models players, game sessions, and AI-controlled characters, with configurable behaviours and session statistics.

## 1.3 Objectives

- Register players and maintain structured digital profiles.
- Generate a unique game session ID for each gameplay session.
- Support adding, viewing, and updating game sessions.
- Demonstrate Java OOPS concepts in a practical project.
- Use role-based permissions and game-event logging.
- Provide a design that can be extended to mobile or web platforms.

## 1.4 Scope

The prototype focuses on player profiles, game-session tracking, rule-based AI decisions, and OOPS design. It is an academic demonstration rather than a production game engine. Real deployment would require performance profiling, gameplay balancing, security review, and extensive playtesting.

## 1.5 Users

| GameCharacter | **Main responsibility** |
|---|---|
| Administrator | Manage player accounts and game settings. |
| AICharacter | View and update authorized player records. |
| Player | View their profile, score, and game progress. |

# 2. EXISTING SYSTEM AND REQUIREMENTS

## 2.1 Existing System

Many games rely on fixed scripts or simple rule-based opponents. Player progress may be stored separately from gameplay logic, making adaptive experiences harder to implement and maintain.

## 2.2 Limitations

- Game progress may be lost when save data is not maintained reliably.
- Static opponent behaviour can make repeated gameplay predictable.
- Different game modes may track player progress inconsistently.
- Manual balancing can create inconsistent difficulty and unfair matches.
- Game events and AI decisions may be difficult to review without logs.

## 2.3 Proposed System

AI + Gaming stores player profiles, game sessions, scores, and AI decision settings in a structured system. During gameplay, the AI module evaluates the current state and selects an action according to the configured strategy. Player-facing features remain separate from game logic, and session events can be logged for review.

## 2.4 Functional Requirements

- Register a player with a unique player ID.
- Create and retrieve a game session using its unique session ID.
- Authenticate users and distinguish roles.

- Create and retrieve game sessions.
- Reject invalid game session IDs and unauthorized access.
- Record key access and update events in an audit log.

### 2.5 Non-Functional Requirements

The system should be maintainable, responsive, usable, and secure. AI decisions should return within acceptable gameplay time, invalid inputs should be rejected, and database queries should use parameterized statements. Player data should be protected, and errors should not expose private information.

### 2.6 Suggested Tools

| Component | Technology |
|---|---|
| Language | Java (JDK 17 or later) |
| Interface | Console prototype; Swing/JavaFX can be added |
| Database | MySQL or another relational database |
| Connectivity | JDBC |
| AI library | Rule-based Java logic |
| IDE | IntelliJ IDEA, Eclipse, or NetBeans |

### 3.2 Workflow

1. The player opens the game and starts a session.
2. The player profile is loaded or a new player is registered.
3. The system initializes the game state and AI configuration.
4. The AI engine evaluates the current game state and available actions.
5. The game logic validates the action and applies the selected AI behaviour.
6. The game updates the world state, score, and player feedback.
7. The session result is logged, then the player can restart or exit.

### 3.3 Textual Architecture Diagram




3.4 Main Use Cases

| Actor | Use cases |
|---|---|
| Administrator | Manage accounts, review audit events. |
| AICharacter | Evaluate game state and select a valid action. |
| Player | View profile, score, preferences, and game history. |
| System | Evaluate game state, validate actions, and log events. |

## 4. OOPS CONCEPTS IN AIGAME

### 4.1 Classes and Objects

A class defines fields and methods for a type of object. For example, Player is a class; each player profile is represented by a Player object. GameSession and NPC are other domain classes.

### 4.2 Encapsulation

Fields are declared private and accessed through methods. This prevents unrelated code from directly changing object state and enables validation to be added in setters or constructors.

### 4.3 Inheritance

A base GameCharacter class can define common identity and behaviour. HumanPlayer and AIPlayer extend it to represent specialized character types. Inheritance avoids duplicating shared fields and methods.

### 4.4 Polymorphism

A method such as decideAction() can be declared in a base AICharacter class and overridden by aggressive, defensive, or balanced AI characters. Calling the method through a base reference selects the implementation belonging to the actual object.

### 4.5 Abstraction

An interface such as GameSessionRepository defines operations without specifying whether data comes from MySQL, a file, or a test implementation. This keeps the service code independent of storage details.

### 4.6 Association and Composition

A Player may be associated with many GameSession objects. A session can contain several AI-controlled characters and events. Related objects can be connected through identifiers and repository queries.

### 4.7 OOPS Mapping Table

| Concept | Example | Benefit |
|---|---|---|
| Class/object | Player, GameSession | Models players, sessions, and AI characters. |
| Encapsulation | private player fields | Protects player and session state. |
| Inheritance | AICharacter extends GameCharacter | Reuses shared behaviour. |
| Polymorphism | decideAction() override | Supports multiple AI strategies. |
| Abstraction | Repository interface | Separates contract from implementation. |
| Composition | Game session with score and event history | Builds larger objects from components. |

## 5. CLASS DESIGN AND DATABASE DESIGN

### 5.1 Main Classes

| Class | Purpose |
|---|---|
| GameCharacter | Common user ID, name, and role behavior. |
| AICharacter | Represents an AI-controlled character. |
| PlayerUser | Represents a player login account. |
| Player | Stores profile, level, and preference details. |
| GameSession | Stores game mode, result, score, and session time. |
| GameAIService | Evaluates game state and selects an AI action. |
| AccessControlService | Checks whether a user can access a record. |
| GameSessionRepository | Defines record persistence operations. |
| AuditLogService | Records access and update events. |

### 5.2 Database Tables

| Table | Important fields |
|---|---|
| users | user_id (PK), username, password_hash, role, status |
| players | player_id (PK), name, player_since, player_level, contact |
| game_sessions | session_id (PK), player_id (FK), ai_character_id, game_mode, score, created_at |
| session_tokens | token_id (PK), player_id (FK), token_hash, expires_at, revoked_at |
| audit_logs | log_id (PK), user_id, player_id, action, timestamp, result |

### 5.3 Relationship Summary

One player can have many game sessions. Each session can contain multiple score and event entries. Player and audit-log relationships help identify which account performed an action.

### 5.4 Example SQL Schema

AI + GAMING — DATABASE SCHEMA

```
CREATE TABLE players (
  player_id VARCHAR(40) PRIMARY KEY,
  player_name VARCHAR(100) NOT NULL,
  player_level VARCHAR(30),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE game_sessions (
  session_id VARCHAR(40) PRIMARY KEY, player_id VARCHAR(40),
  game_mode VARCHAR(40), score INT, ai_action VARCHAR(40)
);
```

## 6. JAVA CODE – DOMAIN CLASSES

The following code illustrates the OOPS domain model for AI + Gaming. Save each public class in its own .java file. This is a learning prototype; production games need stronger validation, persistence, balancing, and anti-cheat controls.

6.1 GameCharacter.java

GameCharacter.java

```
public abstract class GameCharacter {
    private final String characterId;
    private final String name;
    protected GameCharacter(String id, String name) {
        this.characterId = id; this.name = name;
    }
    public String getName() { return name; }
    public abstract String decideAction(int health, int opponentHealth);
}
```

6.2 AICharacterUser.java

```
public class AICharacterUser extends User {
  public AICharacterUser(String userId, String name) {
      super(userId, name);
  }

  @Override
  public String getRole() {
    return "AI_CHARACTER";
  }
}
```

### 6.3 PlayerUser.java

```
public class PlayerUser extends User {
  public PlayerUser(String userId, String name) {
```

```
        super(userId, name);
    }

    @Override
    public String getRole() {
        return "PLAYER";
    }
}
```

**6.4 Player.java**

```
public class Player {
    private final String playerId;
    private String name;
    private String playerLevel;

    public Player(String playerId, String name,
                  String playerLevel) {
        this.playerId = playerId;
        this.name = name;
        this.playerLevel = playerLevel;
    }

    public String getPlayerId() { return playerId; }
    public String getName() { return name; }
    public String getPlayerLevel() { return playerLevel; }

    public void setName(String name) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Name is required");
        this.name = name;
    }

    public void setPlayerLevel(String playerLevel) {
        this.playerLevel = playerLevel;
    }
}
```

**7. JAVA CODE – GAME SESSION AND ABSTRACTION**

**7.1 GameSession.java**

```
import java.time.LocalDateTime;

public class GameSession {
    private final String sessionId;
    private final String playerId;
    private final String aiCharacterId;
    private final String gameMode;
    private final String notes;
    private final LocalDateTime createdAt;

    public GameSession(String sessionId, String playerId,
                       String aiCharacterId, String gameMode,
```

```
                    String notes) {
    this.sessionId = sessionId;
      this.playerId = playerId;
    this.aiCharacterId = aiCharacterId;
      this.gameMode = gameMode;
      this.notes = notes;
      this.createdAt = LocalDateTime.now();
  }

  public String getSessionId() { return sessionId; }
   public String getPlayerId() { return playerId; }
  public String getAICharacterUserId() { return aiCharacterId; }
  public String getGameMode() { return gameMode; }
   public String getNotes() { return notes; }
   public LocalDateTime getCreatedAt() { return createdAt; }
}
```

**7.2 Repository Interface (Abstraction)**

```
import java.util.Optional;

public interface GameSessionRepository {
  void save(GameSession session);
  Optional<GameSession> findBySessionId(String sessionId);
}
```

**7.3 In-Memory Repository for a Simple Demo**

```
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

public class InMemoryGameSessionRepository
      implements GameSessionRepository {
  private final Map<String, GameSession> sessions =
          new HashMap<>();

   @Override
  public void save(GameSession session) {
    sessions.put(session.getSessionId(), session);
  }

   @Override
  public Optional<GameSession> findBySessionId(String id) {
    return Optional.ofNullable(sessions.get(id));
  }
}
```

This repository stores data only while the program runs. Replace it with a JDBC-based implementation for persistent MySQL storage. The interface demonstrates abstraction and allows the game service to be tested without a database.

## 8. JAVA CODE – AI DECISION LOGIC AND GAME STATE

8.1 GameAIService.java

```
public class GameAIService {
  public String chooseAction(int playerHealth, int opponentHealth) {
    if (opponentHealth < 30) return "ATTACK";

    if (playerHealth < 25) return "DEFEND";
    return "BALANCED_MOVE";

    // The selected action is validated by the game rules.

  }
}
```

Important: This is a simplified classroom example. A real game must validate game state, player permissions, server-authoritative actions, and anti-cheat rules. Never trust client-submitted scores or actions without verification.

### 8.2 AI Decision and Game-State Validation

The AI engine should receive structured game-state data, not private account credentials. Validate inputs, constrain available actions, and keep authoritative scoring on the server in multiplayer games.

### 8.3 AIEngineDemo.java (Java AI logic)

This example uses standard Java classes to demonstrate a simple rule-based AI decision. The AI chooses ATTACK when the opponent has low health, DEFEND when the player has low health, and otherwise chooses a balanced move.

```
public class AIEngineDemo {
  public static String chooseAction(int playerHealth, int opponentHealth) {
    if (opponentHealth < 30) return "ATTACK";
    if (playerHealth < 25) return "DEFEND";
    return "BALANCED_MOVE";
  }

  public static void main(String[] args) {
    System.out.println("AI Action: " + chooseAction(80, 20));
  }
}
```

### 8.4 Example Use

```
User aiCharacter = new AICharacterUser("AI100", "Game AI");
User playerAccount = new PlayerUser("P100", "Player One");

System.out.println(aiCharacter.getRole());    // AI_CHARACTER
System.out.println(playerAccount.getRole()); // PLAYER
```

## 9. JAVA CODE – SERVICE AND DEMONSTRATION

### 9.1 AIGameService.java

GameAIService.java

```
public class GameAIService {
    public String chooseAction(int playerHealth, int opponentHealth) {
        if (opponentHealth < 30) return "ATTACK";
        if (playerHealth < 25) return "DEFEND";
        return "BALANCED_MOVE";
    }
}
```

### 9.2 Main.java

Main.java

```
public class Main {
    public static void main(String[] args) {
        GameAIService ai = new GameAIService();
        String action = ai.chooseAction(80, 20);
        System.out.println("AI Action: " + action);
        System.out.println("Game session started.");
    }
}
```

### 9.3 Expected Output

AI GAMEPLAY WORKFLOW

```
1. Load player profile and current game state
2. List valid actions available to the AI character
3. Score actions using simple rules and current health
4. Validate the selected action against game rules
5. Apply action, update score, and log the event
6. Repeat until the game ends
```

## 10. DATABASE CONNECTIVITY AND SECURITY

### 10.1 JDBC Example

The following database layer can use a parameterized SQL insert to store player sessions and scores. Add the MySQL JDBC driver and configure the connection using environment variables or a protected configuration file. Do not hard-code production passwords in source code.

JDBC — SAVE GAME SESSION

```
String sql = "INSERT INTO game_sessions(session_id, player_id, game_mode, score) VALUES (?,
try (Connection con = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, sessionId);
    ps.setString(2, playerId);
    ps.setString(3, gameMode);
    ps.setInt(4, score);
    ps.executeUpdate();
}
```

### 10.2 Security Requirements

- Use HTTPS/TLS for network communication.
- Store passwords using a modern password hash, never plain text.
- Use prepared statements to reduce SQL injection risk.
- Validate AI actions against the current game state and reject impossible or unauthorized moves.
- Expire or revoke tokens when they are no longer valid.
- Authenticate users and enforce authorization on every protected request.
- Limit access to only the player data needed for the task.
- Keep audit logs protected from unauthorized modification.
- Use encrypted backups and periodically test restoration.

### 10.3 Privacy Note

Player data and account information should be handled responsibly. Use fictional data for classroom demonstrations. A real deployment should define privacy, retention, consent, and security policies and undergo testing.

## 11. TESTING AND RESULTS

### 11.1 Test Cases

| No. | Test | Expected result |
|---|---|---|
| 1 | Create Player object | Player profile fields are stored. |
| 2 | Call getRole() on AICharacter | Returns AI_CHARACTER. |
| 3 | Save a game session | Repository stores session. |
| 4 | Find existing session | Correct session is returned. |
| 5 | Find unknown session | No session is returned. |
| 6 | AI character selects a valid action | Operation is accepted in demo. |
| 7 | Player attempts to add record | Operation is rejected. |
| 8 | Player requests another player's record | Access is denied. |
| 9 | Run AI decision engine | A valid game action is selected and displayed. |
| 10 | Insert player using JDBC | Row is saved if DB config/schema are valid. |

### 11.2 Testing Approach

Unit testing should verify individual classes and AI decision methods. Integration testing should verify that the game service, repository, and database work together. Gameplay testing should include invalid actions, unexpected states, score manipulation attempts, and performance under repeated turns.

### 11.3 Results

The sample Java code is designed to demonstrate core OOPS behaviour, game-session storage, and a simple AI action-selection workflow. Run the code in the selected IDE and include actual screenshots of the output before submission.

### 11.4 Acceptance Criteria

- Player and game session objects can be created.
- The service can save and retrieve a session.
- Unauthorized actions are rejected by the service.
- AI decision logic selects an action from the current game state.

Database operations use parameterized statements

## 12. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

### 12.1 Advantages

- Adaptive difficulty can make gameplay more engaging for different skill levels.
- Reusable AI strategies support multiple game modes and opponent styles.
- Structured session and score data is easier to search and maintain.
- OOPS improves modularity, reuse, and maintainability.
- Access checks and audit logs can improve accountability.

The design can be extended to a web or mobile application

## 12.2 Limitations

- The AI prototype uses simplified rules and does not learn from experience.
- The application may depend on network and database availability.
- Incorrect or outdated information can affect usefulness.
- The demo repository does not persist data after the program exits.
- The prototype is not integrated with gaming platform systems.

AI alone cannot guarantee fair play; server-side validation and testing are required.

## 12.3 Future Enhancements

- Build a graphical game interface using JavaFX, Java Swing, or a game engine front end.
- Connect the repository to MySQL through JDBC.
- Add secure login, session expiry, and password hashing.
- Use a cryptographically secure random token with expiry and revocation.
- Add player rankings, achievements, replay analysis, and configurable AI difficulty.
- Explore reinforcement learning and connect the prototype to a multiplayer game server.

Create a mobile version with player profiles and cross-device progress synchronisation

## 12.4 Social and Practical Impact

If implemented and balanced carefully, AI + Gaming may improve replayability, provide adaptive challenges, and make NPC behaviour more engaging. Its success depends on responsive AI decisions, fair rules, player privacy, and thorough testing.

# 13. CONCLUSION AND REFERENCES

## 13.1 Conclusion

AI + Gaming – Intelligent Gameplay and Player Experience System is a suitable Object-Oriented Programming project because it combines a practical gaming use case with classes, objects, encapsulation, inheritance, polymorphism, abstraction, database design, and AI decision logic. The proposed architecture separates interface, game logic, AI decisions, persistence, and security responsibilities.

The Java examples demonstrate the game domain model and a basic session workflow. The AI module selects actions based on the current game state and configured rules. Input validation, server-side score verification, secure authentication, logging, and privacy protections remain essential before real-world use. The prototype can be extended with persistent storage, a graphical interface, and machine-learning models.

## 13.2 References


1. **Oracle. Java Documentation. https://docs.oracle.com/en/java/**
2. **Oracle. The Java Tutorials: Object-Oriented Programming Concepts. https://docs.oracle.com/javase/tutorial/java/concepts/**
3. **MySQL. MySQL Reference Manual. https://dev.mysql.com/doc/**
4. Russell, S. and Norvig, P. Artificial Intelligence: A Modern Approach. https://aima.cs.berkeley.edu/
5. Oracle. Java Documentation. https://docs.oracle.com/en/java/


## 13.3 Execution Notes

Use JDK 17 or later. Save each public Java class in a separate file with the matching filename. Compile the core demo classes together. The JDBC example requires the MySQL connector, a configured database URL, a database user, and the table schema from Section 5.

Suggested submission additions: student name, register number, department, college, guide name, screenshots of the running game-AI demo, and verified test results.